

Homework 3: Recursion

- Due: Thursday 09/24 @ 11:59pm
- Points: 1
- Download: hw03.zip

To receive credit, you must solve each problem and then complete a short checkoff interview about your solution. You may use Preceptor for the interview or come to office hours to be interviewed by a member of the course staff. Staff will be doing in-person checkoffs for up to 3 business days after the assignment deadline (including any approved extension). If a checkoff happens more than 3 business days after the regular deadline, staff may ask to confirm your extended deadline via Flexextensions. You will submit a Provenance bundle that includes a record of how you used VS Code, including interactions with Preceptor. Please do not use AI tools other than Preceptor for this assignment.

Readings: This homework relies on the following readings from Composing Programs:

- 1.7: Recursive Functions

Getting Started Videos:

If you feel stuck, try watching some walkthrough videos. (@berkeley.edu login to YouTube required).

Required Questions

Recursion

Q1: Num Eights

Write a recursive function `num_eights` that takes a positive integer `n` and returns the number of times the digit 8 appears in `n`.

Important: Use recursion; the tests will fail if you use any assignment statements or loops. (You can define new functions, but don't put assignment statements there either.)

```
def num_eights(n):
    """Returns the number of times 8 appears as a digit of n.

    >>> num_eights(3)
    0
    >>> num_eights(8)
    1
    >>> num_eights(88888888)
    8
    >>> num_eights(2638)
    1
    >>> num_eights(86380)
    2
    >>> num_eights(12345)
    0
    >>> num_eights(8782089)
    3
    >>> # This test checks that you used no assignment statements or loops.
    >>> import inspect, ast
    >>> banned = ['Assign', 'AnnAssign', 'AugAssign', 'NamedExpr', 'For', 'While']
    >>> tree = ast.parse(inspect.getsource(num_eights))
    >>> [type(x).__name__ for x in ast.walk(tree) if type(x).__name__ in banned]
    []
    """
    """ YOUR CODE HERE """

python3 -m pytest -k num_eights
```

Q2: Digit Distance

For a given integer, the digit distance is the sum of the absolute differences between consecutive digits. For example:

- The digit distance of 61 is 5, as the absolute value of 6 - 1 is 5.

- The digit distance of 71253 is 12 ($\text{abs}(7-1) + \text{abs}(1-2) + \text{abs}(2-5) + \text{abs}(5-3) = 6 + 1 + 3 + 2$).
- The digit distance of 6 is 0 because there are no pairs of consecutive digits.

Write a function that determines the digit distance of a positive integer. You must use recursion or the tests will fail.

```
def digit_distance(n):
    """Determines the digit distance of n.

    >>> digit_distance(3)
    0
    >>> digit_distance(777) # 0 + 0
    0
    >>> digit_distance(314) # 2 + 3
    5
    >>> digit_distance(31415926535) # 2 + 3 + 3 + 4 + ... + 2
    32
    >>> digit_distance(3464660003) # 1 + 2 + 2 + 2 + ... + 3
    16
    >>> # This test checks that you used no loops.
    >>> import inspect, ast
    >>> tree = ast.parse(inspect.getsource(digit_distance))
    >>> [type(x).__name__ for x in ast.walk(tree) if type(x).__name__ in ('For', 'While')]
    []
    """
    """
    """
    """ YOUR CODE HERE """

python3 -m pytest -k digit_distance
```

Q3: Interleaved Sum

Write a function `interleaved_sum`, which takes in a number `n` and two one-argument functions: `f_odd` and `f_even`. It returns the sum of applying `f_odd` to every odd number and `f_even` to every even number from 1 to `n` inclusive.

For example, executing `interleaved_sum(5, lambda x: x, lambda x: x * x)` returns $1 + 2^2 + 3 + 4^2 + 5 = 29$.

Important: Implement this function without using any loops or directly testing if a number is odd or even (no using `%` or `//` combined with `*`). Instead of directly checking whether a number is even or odd, start with 1, which you know is an odd number.

Hint: Introduce an inner helper function that takes an odd number `k` and computes an interleaved sum from `k` to `n` (including `n`). Alternatively, you can use mutual recursion.

```
def interleaved_sum(n, f_odd, f_even):
    """Compute the sum f_odd(1) + f_even(2) + f_odd(3) + ..., up
    to n.

    >>> identity = lambda x: x
    >>> square = lambda x: x * x
    >>> triple = lambda x: x * 3
    >>> interleaved_sum(5, identity, square) # 1 + 2*2 + 3 + 4*4 + 5
    29
    >>> interleaved_sum(5, square, identity) # 1*1 + 2 + 3*3 + 4 + 5*5
    41
    >>> interleaved_sum(4, triple, square) # 1*3 + 2*2 + 3*3 + 4*4
    32
    >>> interleaved_sum(4, square, triple) # 1*1 + 2*3 + 3*3 + 4*3
    28
    >>> # This test checks that you used no loops, no % (or equivalent workarounds), and
    >>> # no bitwise operators (&, |, ^); don't worry if you don't know what those are.
    >>> import inspect, ast
    >>> banned = ['For', 'While', 'Mod', 'BitAnd', 'BitOr', 'BitXor', 'FloorDiv', 'Mult']
    >>> tree = ast.parse(inspect.getsource(interleaved_sum))
    >>> [type(x).__name__ for x in ast.walk(tree) if type(x).__name__ in banned]
    []
    """
    """
    """
    """ YOUR CODE HERE """

python3 -m pytest -k interleaved_sum
```

Tree Recursion

Q4: Count Coins

Given a positive integer total, a set of coins makes change for total if the sum of the values of the coins is total. Here we will use standard US coin values: 1, 5, 10, and 25. For example, the following sets make change for 15:

- 15 1-cent coins
- 10 1-cent, 1 5-cent coins
- 5 1-cent, 2 5-cent coins
- 5 1-cent, 1 10-cent coins
- 3 5-cent coins
- 1 5-cent, 1 10-cent coins

Thus, there are 6 ways to make change for 15. Write a recursive function `count_coins` that takes a positive integer total and returns the number of ways to make change for total using 1, 5, 10, and 25 cent coins.

Use `next_smaller_coin` in your solution: `next_smaller_coin` will return the next smaller coin value from the input (e.g. `next_smaller_coin(5)` is 1). The function will return `None` if the next coin value does not exist.

Important: Use recursion; the tests will fail if you use loops.

Hint: Refer to the implementation of `count_partitions` for an example of how to count the ways to sum up to a final value with smaller parts. If you need to keep track of more than one value across recursive calls, consider writing a helper function.

```
def next_smaller_coin(coin):
    """Returns the next smaller coin in order."""
    if coin == 25:
        return 10
    elif coin == 10:
        return 5
    elif coin == 5:
        return 1

def count_coins(total):
    """Return the number of ways to make change.

    >>> count_coins(15) # 15 1-cent coins, 10 1-cent & 1 5-cent coins, ... 1 5-cent & 1 10-
cent coins
6
    >>> count_coins(10) # 10 1-cent coins, 5 1-cent & 1 5-cent coins, 2 5-cent coins, 1 10-
cent coin
4
    >>> count_coins(20) # 20 1-cent coins, 15 1-cent & 1 5-cent coins, ... 2 10-cent coins
9
    >>> count_coins(45) # How many ways to make change for 45 cents?
39
    >>> count_coins(100) # How many ways to make change for 100 cents?
242
    >>> count_coins(200) # How many ways to make change for 200 cents?
1463
    >>> # This test checks that you used no loops.
    >>> import inspect, ast
    >>> tree = ast.parse(inspect.getsource(count_coins))
    >>> [type(x).__name__ for x in ast.walk(tree) if type(x).__name__ in ('For', 'While')]
[]
    """
    """ YOUR CODE HERE """

python3 -m pytest -k count_coins
```

Q5: Maximum Subsequence

A subsequence of a number is a series of digits from the number, not necessarily contiguous. For example, 12345 has subsequences like 123, 234, 124, and 245. Implement `max_subseq`, which returns the largest subsequence of `n` that has at most `t` digits. You must use recursion or the tests will fail.

Hint: To add a digit `d` to the end of an existing number `n`, compute `n * 10 + d`. For instance, to add 8 to 15 to get 158, compute `15 * 10 + 8`.

```
def max_subseq(n, t):
    """Return the largest subsequence of at most t digits found in n.
```

For example, for $n = 2012$ and $t = 2$ the subsequences are 2, 0, 1, 2, 20, 21, 22, 01, 02, and 12; the largest is 22.

```
>>> max_subseq(2012, 2)
22
>>> max_subseq(20125, 3)
225
>>> max_subseq(20125, 5)
20125
>>> max_subseq(20125, 6) # note that 20125 == 020125
20125
>>> max_subseq(12345, 3)
345
>>> max_subseq(12345, 0) # 0 is of length 0
0
>>> max_subseq(12345, 1)
5
>>> # This test checks that you used no loops.
>>> import inspect, ast
>>> tree = ast.parse(inspect.getsource(max_subseq))
>>> [type(x).__name__ for x in ast.walk(tree) if type(x).__name__ in ('For', 'While')]
[]
"""
**** YOUR CODE HERE ****
```

```
python3 -m pytest -k max_subseq
```

Split into the case where the ones digit of n is used and the case where it is not. When it is used, t decreases by one because a digit was spent; when it isn't, t stays the same. The answer is the larger of the two.

Submit

Run Provenance: Prepare Submission Bundle from the VS Code command palette to create your submission zip, and upload that zip to Gradescope. For a refresher on how to do this, refer to Lab 00.

Exam Practice

Homework assignments will also contain prior exam-level questions for you to take a look at. These questions have no submission component; feel free to attempt them if you'd like a challenge!

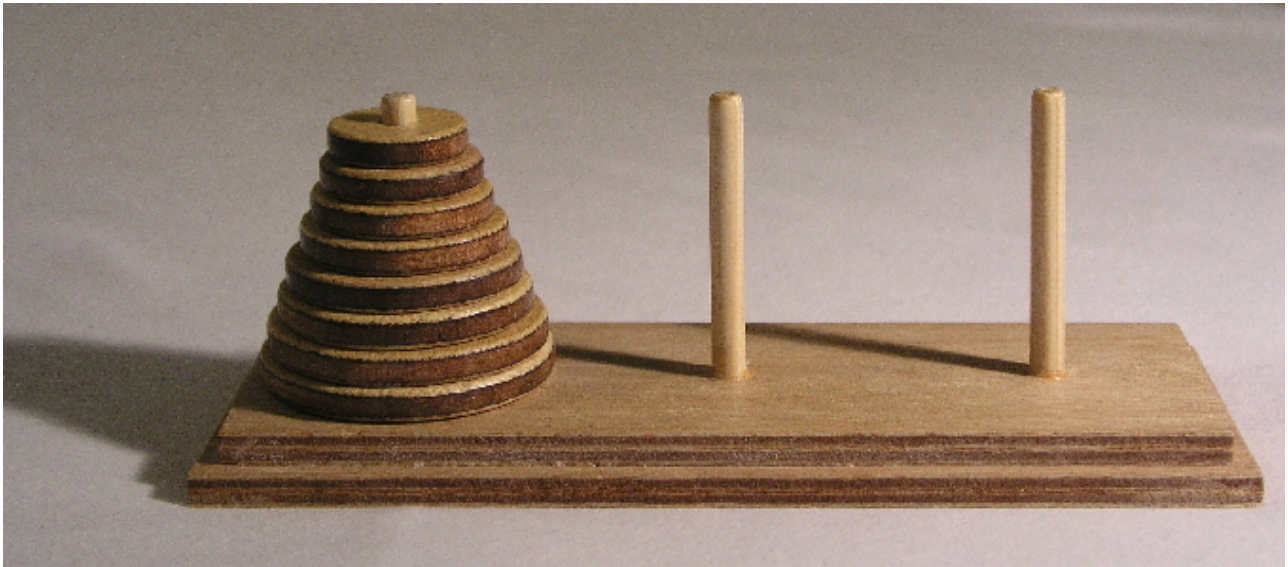
- Fall 2017 MT1 Q4a: Digital
- Fall 2019 Final Q6b: Palindromes

Just For Fun Questions

The questions below are out of scope for 61A. You can try them if you want an extra challenge, but they're just puzzles that are not required for the course. Almost all students will skip them, and that's fine. We will not be prioritizing support for these questions on Ed or during Office Hours.

Q6: Towers of Hanoi

A classic puzzle called the Towers of Hanoi is a game that consists of three rods, and a number of disks of different sizes which can slide onto any rod. The puzzle starts with n disks in a neat stack in ascending order of size on a start rod, the smallest at the top, forming a conical shape.



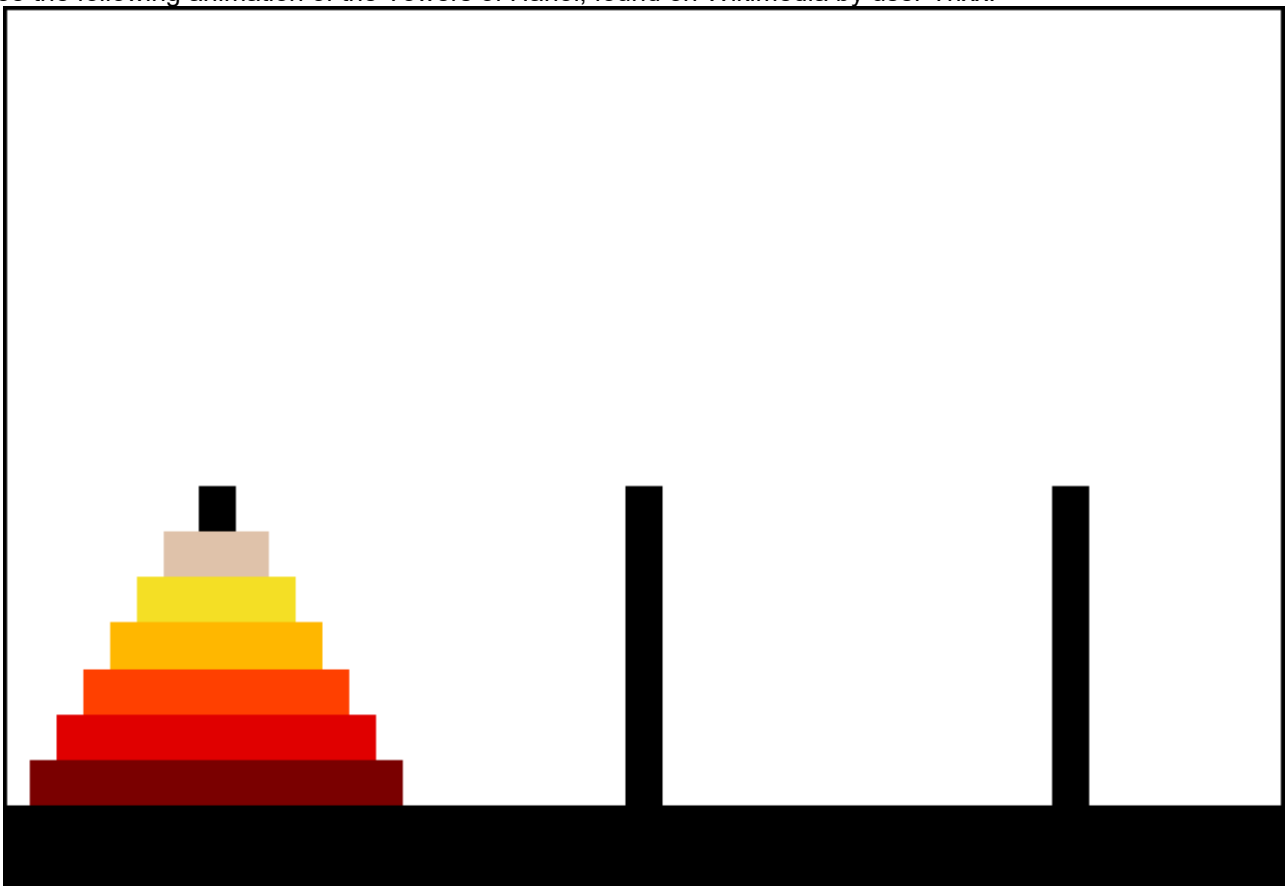
The objective of the puzzle is to move the entire stack to an end rod, obeying the following rules:

- Only one disk may be moved at a time.
- Each move consists of taking the top (smallest) disk from one of the rods and sliding it onto another rod, on top of the other disks that may already be present on that rod.
- No disk may be placed on top of a smaller disk.

Complete the definition of `move_stack`, which prints out the steps required to move n disks from the start rod to the end rod without violating the rules. The provided `print_move` function will print out the step to move a single disk from the given origin to the given destination.

Hint: Draw out a few games with various n on a piece of paper and try to find a pattern of disk movements that applies to any n . In your solution, take the recursive leap of faith whenever you need to move any amount of disks less than n from one rod to another. If you need more help, see the following hints.

See the following animation of the Towers of Hanoi, found on Wikimedia by user Trixx.



The strategy used in Towers of Hanoi is to move all but the bottom disc to the second peg, then moving the bottom disc to the third peg, then moving all but the second disc from the second to the third peg.

One thing you don't need to worry about is collecting all the steps. print effectively "collects" all the results in the terminal as long as you make sure that the moves are printed in order.

```
def print_move(origin, destination):
    """Print instructions to move a disk."""
    print("Move the top disk from rod", origin, "to rod", destination)

def move_stack(n, start, end):
    """Print the moves required to move n disks on the start pole to the end
    pole without violating the rules of Towers of Hanoi.

    n -- number of disks
    start -- a pole position, either 1, 2, or 3
    end -- a pole position, either 1, 2, or 3

    There are exactly three poles, and start and end must be different. Assume
    that the start pole has at least n disks of increasing size, and the end
    pole is either empty or has a top disk larger than the top n start disks.

    >>> move_stack(1, 1, 3)
    Move the top disk from rod 1 to rod 3
    >>> move_stack(2, 1, 3)
    Move the top disk from rod 1 to rod 2
    Move the top disk from rod 1 to rod 3
    Move the top disk from rod 2 to rod 3
    >>> move_stack(3, 1, 3)
    Move the top disk from rod 1 to rod 3
    Move the top disk from rod 1 to rod 2
    Move the top disk from rod 3 to rod 2
    Move the top disk from rod 1 to rod 3
    Move the top disk from rod 2 to rod 1
    Move the top disk from rod 2 to rod 3
    Move the top disk from rod 1 to rod 3
    """
    assert 1 <= start <= 3 and 1 <= end <= 3 and start != end, "Bad start/end"
    """ YOUR CODE HERE """

python3 -m pytest -k move_stack
```

Q7: Anonymous Factorial

This question demonstrates that it's possible to write recursive functions without assigning them a name in the global frame.

The recursive factorial function can be written as a single expression by using a conditional expression.

```
>>> fact = lambda n: 1 if n == 1 else mul(n, fact(sub(n, 1)))
>>> fact(5)
120
```

However, this implementation relies on the fact (no pun intended) that fact has a name, to which we refer in the body of fact. To write a recursive function, we have always given it a name using a def or assignment statement so that we can refer to the function within its own body. In this question, your job is to define fact recursively without using a def or assignment statement to give it a name.

Write an expression that computes n factorial using only call expressions, conditional expressions, and lambda expressions (no assignment or def statements).

Note: You are not allowed to use make_anonymous_factorial in your return expression.

The sub and mul functions from the operator module are the only built-in functions required to solve this problem.

Hint: In order to recursively compute the factorial, you need a name to use in the recursive calls. What other ways have we learned to give something a name, besides assignment statements and def statements?

```
from operator import sub, mul

def make_anonymous_factorial():
    """Return the value of an expression that computes factorial.

    >>> make_anonymous_factorial()(5)
    120
```

```
>>> # This test checks that the body is just a return statement that
>>> # doesn't refer to make_anonymous_factorial.
>>> import inspect, ast
>>> body = ast.parse(inspect.getsource(make_anonymous_factorial)).body[0].body
>>> [type(x).__name__ for x in body]
['Expr', 'Return']
>>> 'make_anonymous_factorial' in ast.dump(body[-1])
False
"""
return 'YOUR_EXPRESSION_HERE'

python3 -m pytest -k make_anonymous_factorial
```